

Tarea 2 — Fundamentos de Ciencias de la Computación

Semestre Otoño 2026

Reproductor MP3 con Sincronización de Letras (.lrc)

Profesores: Luis Gajardo – Martita Muñoz

1. Descripción General

Esta tarea implementa un reproductor de música MP3 que sincroniza las letras de las canciones (.lrc) en tiempo real. El sistema se divide en tres etapas: (1) diseño e implementación de la gramática LR para el formato .lrc usando SableCC, (2) implementación del visitador que construye el repositorio de letras en memoria, y (3) integración con el reproductor MP3 usando un Timer para la sincronización.

2. Formato de Archivos Lyrics (.lrc)

Los archivos .lrc son archivos de texto plano que contienen el texto de una canción sincronizado con marcas de tiempo. Cada línea sigue el formato:

Línea de lyric: [mm:ss.xx]texto de la canción

Los archivos también pueden incluir metadatos al inicio:

Etiqueta	Significado	Ejemplo
[ar:...]	Artista	[ar:Metallica]
[ti:...]	Título	[ti:Nothing Else Matters]
[al:...]	Álbum	[al:Metallica]
[length:]	Duración	[length:06:28]
[xx:...]	Otros metadatos	[id:pshjuazd]

3. Diseño de la Gramática (Etapa 1)

Se diseñó una gramática libre de contexto para ser usada con el parser LR de SableCC. La gramática reconoce el formato .lrc completo incluyendo metadatos y líneas de lyric.

3.1 Tokens (análisis léxico)

Token	Patrón	Descripción
time_tag	[dd:dd.dd]	Marca de tiempo
tag_ar	[ar:texto]	Metadato artista
tag_ti	[ti:texto]	Metadato título
tag_al	[al:texto]	Metadato álbum
tag_length	[length:texto]	Metadato duración
tag_other	[xx:texto]	Otros metadatos
lyric_text	line_char+	Texto de la canción
eol	(cr lf) cr lf	Fin de línea
blank	(tab)+	Espacios (ignorados)

3.2 Producciones (análisis sintáctico)

Las producciones definen la estructura jerárquica del archivo .lrc:

```
archivo  → metadata lineas      /* con metadatos */
          | lineas              /* sin metadatos */

metadata → meta_tag eol        /* un metadato */
          | meta_tag eol metadata /* varios */

meta_tag → tag_ar              /* artista   */
          | tag_ti              /* título   */
          | tag_al              /* álbum   */
          | tag_length          /* duración */
          | tag_other           /* otros    */

lineas   → linea               /* una línea */
          | linea lineas       /* varias   */

linea    → time_tag lyric_text eol /* con texto */
          | time_tag eol          /* vacía    */
```

3.3 Árbol de derivación

Para la entrada [ar:Metallica]\n[01:00.32]So close, no matter how far, el árbol de derivación generado es:

```

■ ■ archivo
  ■ ■ metadata
    ■ ■ meta_tag → tag_ar : "Metallica"
  ■ ■ lineas
    ■ ■ linea
      ■ ■ time_tag → [01:00.32]
      ■ ■ lyric_text → "So close, no matter how far"

```

El árbol completo de cada archivo se visualiza en tiempo real en el panel inferior de la interfaz gráfica del reproductor.

4. Implementación del Visitador (Etapas 2)

El visitador implementado en `LrcParser.java` recorre el árbol de derivación producido por la gramática y extrae la información relevante, almacenándola en un `ArrayList<LyricItem>` (repositorio en memoria principal).

4.1 Clase LyricItem

Cada objeto `LyricItem` almacena:

Campo	Tipo	Descripción
timeMs	long	Tiempo en milisegundos (calculado desde mm:ss.xx)
text	String	Texto del lyric en ese instante

La fórmula de conversión de tiempo es:

$$\text{timeMs} = (\text{mm} * 60 + \text{ss}) * 1000 + \text{xx} * 10$$

4.2 Proceso del visitador

El método `parse(String path)` de `LrcParser` realiza las siguientes etapas:

1. Leer todas las líneas del archivo `.lrc` en memoria.
2. Invocar `parseMetadata()` para identificar y consumir las etiquetas de metadatos (`[ar:]`, `[ti:]`, `[al:]`, `[length:]`, etc.) al inicio del archivo.
3. Invocar `parseLineas()` para procesar el cuerpo del archivo: cada línea con marca de tiempo `[mm:ss.xx]` genera un `LyricItem` en el repositorio.
4. Construir el árbol de derivación en texto para su visualización en la GUI.

Al finalizar el recorrido, el `ArrayList` contiene todos los lyrics ordenados cronológicamente, listos para la sincronización con el reproductor.

5. Integración al Reproductor (Etapa 3)

La clase `Reproductor.java` integra el reproductor MP3 (usando la librería `BasicPlayer`) con el sistema de sincronización de letras.

5.1 Clase Reminder (Timer interno)

Se implementó la clase interna abstracta `Reminder` que encapsula un `java.util.Timer`. Al presionar Play, se programa un `Reminder` por cada `LyricItem` del repositorio, calculando el delay como la diferencia entre el timestamp del ítem y el tiempo ya reproducido:

```
delay = lyricItem.getTimeMs() - elapsedMs

new Reminder(delay, item) {
    public void onTime() { showLyric(idx); }
};
```

5.2 Manejo de Pausa y Reanudación

Al pausar, se cancela el `Timer` global y se registra el tiempo acumulado (`pausedAt`). Al reanudar, se reprograman todos los `Reminders` restantes usando `scheduleAllReminders(pausedAt)`, de forma que la sincronización continúa exactamente donde se dejó.

5.3 Interfaz Gráfica (GUI)

La GUI implementada con `Swing` incluye:

Panel superior: Título, artista/álbum y contador de tiempo en curso.

Panel central: Línea actual del lyric en verde (grande) y línea siguiente en gris tenue.

Panel árbol: Árbol de derivación completo del archivo `.lrc` cargado.

Panel inferior: Carga de archivos MP3 y LRC (por botón o arrastrar), y controles Play / Pausa / Stop.

Carga automática: Al abrir un `.mp3`, si existe un `.lrc` con el mismo nombre en la misma carpeta, se carga automáticamente.

6. Archivos de Prueba

Archivo MP3	Archivo LRC	Artista	Metadatos presentes
metallica.mp3	nothing_else_matters.lrc	Metallica	[ar], [al], [ti], [length], [id]
evanescence.mp3	bring_me_to_life.lrc	Evanescence	[ar], [ti], [by], [re], [ve], [id]

7. Compilación y Ejecución

Requiere: Java JDK 11 o superior. Los JARs de BasicPlayer están incluidos en libs/.

Compilar:

```
chmod +x compile.sh run.sh
./compile.sh
```

Ejecutar (con argumentos):

```
./run.sh metallica.mp3 nothing_else_matters.lrc
./run.sh evanescence.mp3 bring_me_to_life.lrc
```

Ejecutar (modo GUI, sin argumentos):

```
./run.sh
# Usar botones 'Abrir MP3' y 'Abrir LRC' en la interfaz
```

Para regenerar el parser con SableCC (requiere sablecc.jar):

```
java -jar sablecc.jar lyrics/lrc.sablecc
# Genera: lyrics/lexer/, lyrics/parser/, lyrics/node/, lyrics/analysis/
# Luego recompilar con ./compile.sh
```